

INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

GRC102 - Information Security Governance

Module 4 - Monitoring and Auditing Security Controls

Week 4 Practical Laboratory

Linux Security Monitoring and Auditing

From Technical Evidence to Governance Assurance

Prepared by	Oluwatosin Olajubu
Registration number	C11/CGRCE/26/217602
Role	Security Control Assurance Analyst (assigned scenario role)
Environment	My personal Kali Linux, My ICDFA Ubuntu host was down
Report date	30 September 2026

A note on how I wrote this: everything below is written the way I actually did the work - I ran a command, looked at what came back, and worked out what it meant. Every screenshot is from my own assigned VM, and I have said clearly anywhere I wasn't 100% sure about something.

Contents

Section	Content
1	Executive Summary
2	Scope and Authorisation
3	Methodology
4	Module 1 Findings - Setting Up and Using auditd
5	Module 2 Findings - Looking Through the Logs
6	Module 3 Findings - Running Lynis
7	Control-Monitoring Table
8	How This Would Feed a SIEM
9	Answering the Governance Questions
10	Remediation and Retest Plan
11	Conclusion
Appendix A	Finding Records
Appendix B	Screenshot Index

1. Executive Summary

I had issues activating my ICDFA Ubuntu environment saying something like the host was down. Even when I tried the SSH connection, still was not working so i had to use my Kali Linux VM and acted as a Security Control Assurance Analyst: set up auditing, look through the logs, run a security scan, and work out what any of it actually means from a governance point of view - not just show that commands ran.

The short version of what I found: the basic building blocks work once set up, but a few things that looked fine on the surface were not actually doing their job yet. I realised auditd was not installed to start with, and the rule meant to catch authentication failures had never actually caught one. A security scan showed there is no firewall, no intrusion detection, and no malware scanner on this machine at all, and quite a few kernel and network settings did not match a hardened baseline. I also found a service (sshd's generator) crashing repeatedly in the logs, which nobody had looked at.

My top three findings, in order of how urgent I think they are:

- **No firewall, intrusion detection, or malware scanner is installed.** This is the biggest gap, because right now there is basically nothing actively watching or blocking anything on this host.
- **The rule I set up to catch authentication failures has never actually fired on a real failure** - every event under that key so far is just the rule being created, not a login failing. So I can't yet prove this control would catch a real brute-force attempt.
- **A system service (systemd-sshd-generator) has crashed with a segfault on at least two separate boots,** and nobody had noticed until I went looking through the error-priority logs.

Meanwhile, on the bright side: once auditd was installed, it came up and stayed running, the rules loaded correctly, and I could see exactly who ran what and when. The hardening scan came out at 61 out of 100, and when I fixed one small thing myself (installing apt-listchanges), I could prove the fix worked by re-running the scan and seeing the suggestion disappear - the same close-the-loop approach I'd want applied to everything else I flag below.

2. Scope and Authorisation

Everything in this report was done on my personal Kali Linux VM (hostname kali, user kali). I did not touch, scan, or try to log into any other machine, network, or account because this had to stay inside my own VM, and that is where I stayed.

Item	Detail
Environment	Kali Linux (rolling release), hostname kali, user kali
Evidence time window	Mostly 28 September to 1 October 2026, plus some older log history going back to July that was already on the VM from previous boots
Tools I used	auditd / auditctl / ausearch / aureport, journalctl and grep, and Lynis 3.1.6
What I did not do	I didn't attempt any real authentication attack, didn't change /etc/passwd or /etc/shadow (I opened /etc/passwd in nano and closed it without saving), and didn't touch anything outside this VM

3. Methodology

I worked through the lab in order: get auditd working and generate some events, then look at the logs a different way with journalctl and grep, then run a hardening scan to see how the system looks. After each step I asked myself the same question: does this tell me anything a manager or auditor would actually want to know? That is the thinking behind the findings, the control table, and the remediation plan later in this report.

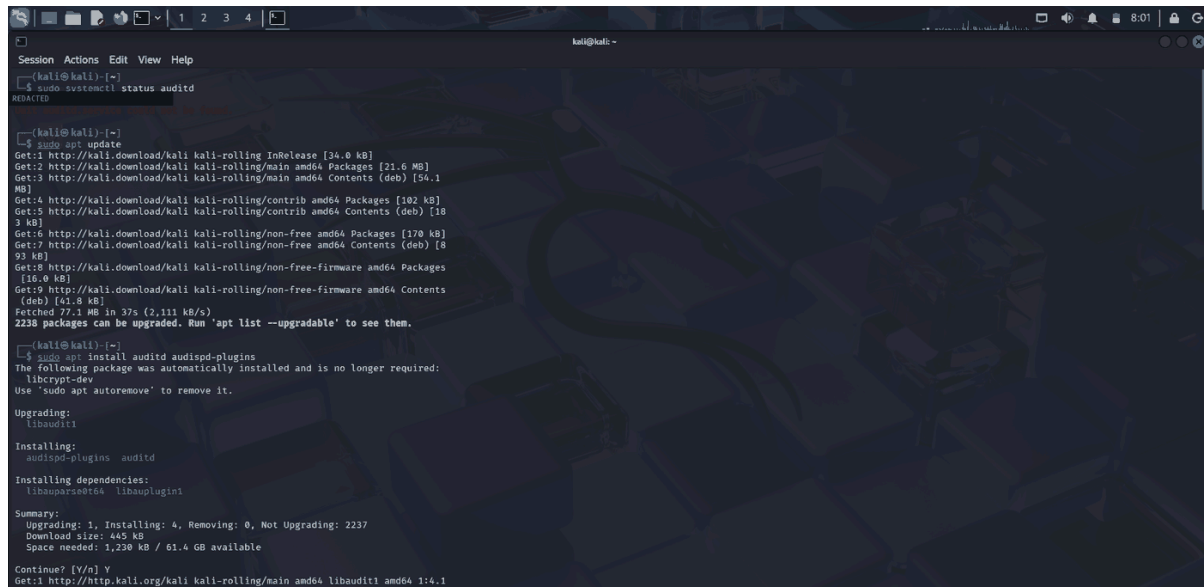
Step	What I did
1. auditd	Checked if it was installed, installed it, confirmed it was running, wrote five custom rules, restarted the service, generated a harmless test event, and queried the results with ausearch and aureport.
2. Logs	Used journalctl a few different ways (general, by unit, by time, by priority, live-following), then grepped for failed passwords, sudo activity, errors and warnings.
3. Hardening scan	Ran a full system scan with Lynis, read through the warnings and suggestions, picked one low-risk suggestion (installing apt-listchanges), fixed it, and re-ran the scan to prove it worked.
4. Governance	Took the evidence from steps 1-3 and worked out: who owns each issue, how bad it is, what a monitoring system would do with it, and what I'd want to see before calling it fixed.

4. Setting Up and Using auditd

4.1 Checking if auditd was even there

First thing I did was run `sudo systemctl status auditd` to see if the audit service already existed. It didn't - I got "Unit `auditd.service` could not be found." So `auditd` wasn't installed by default on this VM, which I believe is worth noting on its own because if someone assumed this control was already running, they would be wrong.

Figure 1: Checking for `auditd`, then installing it



```
Session Actions Edit View Help
(kali@kali) ~
$ sudo systemctl status auditd
Unit auditd.service could not be found.

(kali@kali) ~
$ sudo apt update
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [21.0 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [54.1 MB]
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [102 kB]
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [18 3 kB]
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [170 kB]
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [8 93 kB]
Get:8 http://kali.download/kali kali-rolling/non-free-firmware amd64 Packages [16.0 kB]
Get:9 http://kali.download/kali kali-rolling/non-free-firmware amd64 Contents (deb) [41.8 kB]
Fetched 77.1 MB in 37s (2,111 kB/s)
2238 packages can be upgraded. Run 'apt list --upgradable' to see them.

(kali@kali) ~
$ sudo apt install auditd audispd-plugins
The following package was automatically installed and is no longer required:
  libcrypt-dev
Use 'sudo apt autoremove' to remove it.

Upgrading:
  libaudit1

Installing:
  audispd-plugins auditd

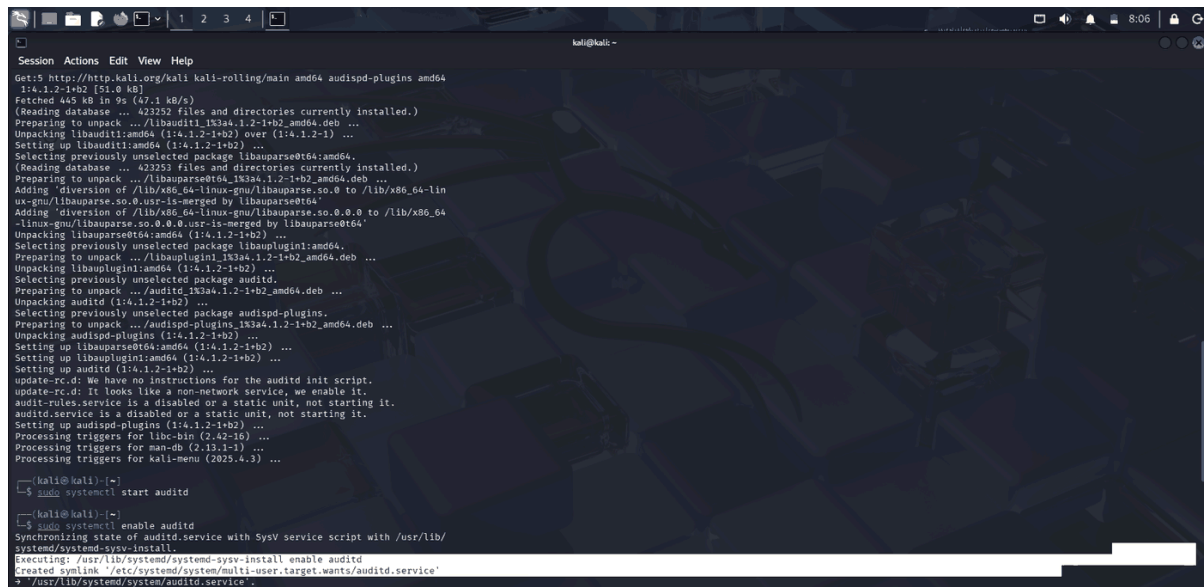
Installing dependencies:
  libauparse0t64 libauplugin1

Summary:
  Upgrading: 1, Installing: 4, Removing: 0, Not Upgrading: 2237
  Download size: 445 kB
  Space needed: 1,238 kB / 61.4 GB available

Continue? [Y/n] Y
Get:1 http://http.kali.org/kali kali-rolling/main amd64 libaudit1 amd64 1:4.1
```

systemctl status auditd comes back saying the unit doesn't exist. I then ran `apt update` and `apt install auditd audispd-plugins` to install it.

Figure 2: Finishing the install and turning it on



```
Session Actions Edit View Help
(kali@kali) ~
$ sudo systemctl start auditd
Unit auditd.service is starting.
Unit auditd.service started.

(kali@kali) ~
$ sudo systemctl enable auditd
Synchronizing state of auditd.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable auditd
Created symlink /etc/systemd/system/multi-user.target.wants/auditd.service
→ /usr/lib/systemd/system/auditd.service.
```

apt finishes setting up auditd and its dependencies. I then ran `systemctl start auditd` and `systemctl enable auditd` - the enable command shows it created a symlink so the service will start automatically from now on.

Figure 3: Confirming it is actually active

```
(kali㉿kali)-[~]
$ systemctl status auditd
● auditd.service - Security Audit Logging Service
   Loaded: loaded (/usr/lib/systemd/system/auditd.service; enabled; preset: disable>
   Active: active (running) since Mon 2026-09-28 08:21:04 EDT; 3 days ago
     Invocation: 8cc1091059d14felaa5fe93e9f4dcb23
       Docs: man:auditd(8)
            https://github.com/linux-audit/audit-documentation
    Main PID: 15526 (auditd)
      Tasks: 2 (limit: 2105)
     Memory: 41.7M (peak: 42.6M)
        CPU: 1min 4.228s
    CGroup: /system.slice/auditd.service
            └─15526 /usr/sbin/auditd

Sep 29 18:02:59 kali auditd[15526]: Audit daemon rotating log files
Sep 29 18:03:13 kali auditd[15526]: Audit daemon rotating log files
Sep 29 18:03:33 kali auditd[15526]: Audit daemon rotating log files
Sep 29 18:03:49 kali auditd[15526]: Audit daemon rotating log files
Sep 29 18:04:05 kali auditd[15526]: Audit daemon rotating log files
Sep 29 18:05:17 kali auditd[15526]: Audit daemon rotating log files
Sep 29 18:06:09 kali auditd[15526]: Audit daemon rotating log files
Sep 29 18:07:46 kali auditd[15526]: Audit daemon rotating log files
Oct 01 12:03:32 kali auditd[15526]: Audit daemon rotating log files
Oct 01 12:03:51 kali auditd[15526]: Audit daemon rotating log files
lines 1-23/23 (END)
```

A few days later I went back and ran `systemctl status auditd` again on its own, just to be sure. It shows `Active: active (running)` since Mon 2026-09-28 08:21:04, so the service has been up continuously since I installed it, and the log shows it quietly rotating its own log files every so often in the background.

4.2 Writing the audit rules

Rather than editing the main rules file, I created a new one at `/etc/audit/rules.d/custom.rules` and put five rules in it:

Rule	What it's for
<code>-w /etc/passwd -p rwx -k passwd_changes</code>	Watch the main account file for any access or change
<code>-w /etc/shadow -p rwx -k shadow_changes</code>	Watch the password-hash file the same way
<code>-a always,exit -F arch=b64 -S execve -k program_execution</code>	Log every program that gets run (64-bit)
<code>-a always,exit -F arch=b32 -S execve -k program_execution</code>	Same thing for 32-bit programs
<code>-w /var/log/auth.log -p wa -k auth_failures</code>	Watch the authentication log file for writes or changes

Figure 4: Typing the rules into nano

```
GNU nano 8.6 /etc/audit/rules.d/custom.rules
-w /etc/passwd -p rwx -k passwd_changes
-w /etc/shadow -p rwx -k shadow_changes
-a always,exit -F arch=b64 -S execve -k program_execution
-a always,exit -F arch=b32 -S execve -k program_execution
-w /var/log/auth.log -p wa -k auth_failures
```

All five rules, typed into `/etc/audit/rules.d/custom.rules` before saving.

Figure 5: Confirming the file saved correctly

```
Session Actions Edit View Help
Selecting previously unselected package auditd-plugins.
Preparing to unpack .../auditd-plugins_1%3a4.1.2-1+b2_amd64.deb ...
Unpacking auditd-plugins (1:4.1.2-1+b2) ...
Setting up libaudit-common:amd64 (1:4.1.2-1+b2) ...
Setting up libaudit1:amd64 (1:4.1.2-1+b2) ...
Setting up auditd (1:4.1.2-1+b2) ...
update-rc.d: We have no instructions for the auditd init script.
update-rc.d: It looks like a non-network service, we enable it.
audit-rules.service is a disabled or a static unit, not starting it.
auditd.service is a disabled or a static unit, not starting it.
Setting up auditd-plugins (1:4.1.2-1+b2) ...
Processing triggers for libc-bin (2.42-16) ...
Processing triggers for man-db (2.13.1-1) ...
Processing triggers for kali-menu (2025.4.3) ...

(kali@kali)-[~]
$ sudo systemctl start auditd

(kali@kali)-[~]
$ sudo systemctl enable auditd
Synchronizing state of auditd.service with SysV service script with /usr/lib/
systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable auditd
Created symlink '/etc/systemd/system/multi-user.target.wants/auditd.service'
→ '/usr/lib/systemd/system/auditd.service'.

(kali@kali)-[~]
$ sudo systemctl start auditd

(kali@kali)-[~]
$ sudo nano /etc/audit/rules.d/custom.rules

(kali@kali)-[~]
$ sudo cat /etc/audit/rules.d/custom.rules
-w /etc/passwd -p rwx -k passwd_changes
-w /etc/shadow -p rwx -k shadow_changes
-a always,exit -F arch=b64 -S execve -k program_execution
-a always,exit -F arch=b32 -S execve -k program_execution
-w /var/log/auth.log -p wa -k auth_failures

(kali@kali)-[~]
$
```

Running cat on the rules file shows the same five lines came back, so the save worked.

Figure 6: Restarting auditd and checking the rules loaded

```
Session Actions Edit View Help
update-rc.d: It looks like a non-network service, we enable it.
audit-rules.service is a disabled or a static unit, not starting it.
auditd.service is a disabled or a static unit, not starting it.
Setting up auditd-plugins (1:4.1.2-1+b2) ...
Processing triggers for libc-bin (2.42-16) ...
Processing triggers for man-db (2.13.1-1) ...
Processing triggers for kali-menu (2025.4.3) ...

(kali@kali)-[~]
$ sudo systemctl start auditd

(kali@kali)-[~]
$ sudo systemctl enable auditd
Synchronizing state of auditd.service with SysV service script with /usr/lib/
systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable auditd
Created symlink '/etc/systemd/system/multi-user.target.wants/auditd.service'
→ '/usr/lib/systemd/system/auditd.service'.

(kali@kali)-[~]
$ sudo systemctl start auditd

(kali@kali)-[~]
$ sudo nano /etc/audit/rules.d/custom.rules

(kali@kali)-[~]
$ sudo cat /etc/audit/rules.d/custom.rules
-w /etc/passwd -p rwx -k passwd_changes
-w /etc/shadow -p rwx -k shadow_changes
-a always,exit -F arch=b64 -S execve -k program_execution
-a always,exit -F arch=b32 -S execve -k program_execution
-w /var/log/auth.log -p wa -k auth_failures

(kali@kali)-[~]
$ sudo systemctl restart auditd

(kali@kali)-[~]
$ sudo auditctl -l
-w /etc/passwd -p rwx -k passwd_changes
-w /etc/shadow -p rwx -k shadow_changes
-a always,exit -F arch=b64 -S execve -F key=program_execution
-a always,exit -F arch=b32 -S execve -F key=program_execution
-w /var/log/auth.log -p wa -k auth_failures

(kali@kali)-[~]
$
```

After systemctl restart auditd, auditctl -l lists all five rules as active in the kernel. The two program_execution rules now print as -F key=program_execution - that is just auditctl's own way of showing the same -k option back to me.

4.3 Generating a test event and searching for it

I wanted a harmless test event, so I opened /etc/passwd in nano and closed it again without saving anything, then ran ls /tmp straight after.

Figure 7: A harmless test event

```

kali@kali: ~
$ sudo systemctl enable auditd
Synchronizing state of auditd.service with SysV service script with /usr/lib/
systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable auditd
Created symlink /etc/systemd/system/multi-user.target.wants/auditd.service'
→ /usr/lib/systemd/system/auditd.service'.

(kali@kali)-[~]
$ sudo systemctl start auditd

(kali@kali)-[~]
$ sudo nano /etc/audit/rules.d/custom.rules

(kali@kali)-[~]
$ sudo cat /etc/audit/rules.d/custom.rules
-w /etc/passwd -p rwx -k passwd_changes
-w /etc/shadow -p rwx -k shadow_changes
-a always,exit -F arch=b64 -S execve -k program_execution
-a always,exit -F arch=b32 -S execve -k program_execution
-w /var/log/auth.log -p wa -k auth_failures

(kali@kali)-[~]
$ sudo systemctl restart auditd

(kali@kali)-[~]
$ sudo auditctl -l
-w /etc/passwd -p rwx -k passwd_changes
-w /etc/shadow -p rwx -k shadow_changes
-a always,exit -F arch=b64 -S execve -F key=program_execution
-a always,exit -F arch=b32 -S execve -F key=program_execution
-w /var/log/auth.log -p wa -k auth_failures

(kali@kali)-[~]
$ sudo nano /etc/passwd

(kali@kali)-[~]
$ ls /tmp
config-err-b6lv3a                                systemd-private-65677961d774f42ba20fd583c260d7-colord.service-8zn5P9
systemd-private-65677961d774f42ba20fd583c260d7-havaged.service-30Y9w
systemd-private-65677961d774f42ba20fd583c260d7-ModemManager.service-inLRh
systemd-private-65677961d774f42ba20fd583c260d7-polkit.service-x9Qv2E
systemd-private-65677961d774f42ba20fd583c260d7-systemd-logind.service-3QuAAW
systemd-private-65677961d774f42ba20fd583c260d7-upower.service-9K8QkB
vmtoolsd-root_564-2965382487

```

sudo nano /etc/passwd (opened and closed with no changes saved), then ls /tmp right after.

Figure 8: ausearch -k passwd_changes

```

(kali@kali)-[~]
$ sudo ausearch -k passwd_changes

time→Mon Sep 28 08:21:04 2026
type=PROCTITLE msg=audit(1790598064.665:173): proctitle=2F7362696E2F617564697463746C00
type=SYSCALL msg=audit(1790598064.665:173): arch=c000003e syscall=44 success=yes exit=
gid=0 fsgid=0 tty=(none) ses=4294967295 comm="auditctl" exe="/usr/sbin/auditctl" subj=
type=CONFIG_CHANGE msg=audit(1790598064.665:173): auid=4294967295 ses=4294967295 subj=

time→Mon Sep 28 08:21:33 2026
type=PROCTITLE msg=audit(1790598093.733:355): proctitle=7375646F00617564697463746C002D
type=PATH msg=audit(1790598093.733:355): item=0 name="/etc/passwd" inode=132815 dev=08
type=CWD msg=audit(1790598093.733:355): cwd="/home/kali"
type=SYSCALL msg=audit(1790598093.733:355): arch=c000003e syscall=257 success=yes exit
id=1000 sgid=1000 fsgid=1000 tty=pts0 ses=2 comm="sudo" exe="/usr/bin/sudo" subj=uncon

time→Mon Sep 28 08:21:33 2026
type=PROCTITLE msg=audit(1790598093.733:356): proctitle=7375646F00617564697463746C002D
type=PATH msg=audit(1790598093.733:356): item=0 name="/etc/passwd" inode=132815 dev=08
type=CWD msg=audit(1790598093.733:356): cwd="/home/kali"
type=SYSCALL msg=audit(1790598093.733:356): arch=c000003e syscall=257 success=yes exit
id=1000 sgid=1000 fsgid=1000 tty=pts0 ses=2 comm="sudo" exe="/usr/bin/sudo" subj=uncon

time→Mon Sep 28 08:21:33 2026
type=PROCTITLE msg=audit(1790598093.737:357): proctitle=7375646F00617564697463746C002D
type=PATH msg=audit(1790598093.737:357): item=0 name="/etc/passwd" inode=132815 dev=08
type=CWD msg=audit(1790598093.737:357): cwd="/home/kali"
type=SYSCALL msg=audit(1790598093.737:357): arch=c000003e syscall=257 success=yes exit
id=1000 sgid=1000 fsgid=1000 tty=pts0 ses=2 comm="sudo" exe="/usr/bin/sudo" subj=uncon

time→Mon Sep 28 08:21:33 2026
type=PROCTITLE msg=audit(1790598093.745:358): proctitle=7375646F00617564697463746C002D
type=PATH msg=audit(1790598093.745:358): item=0 name="/etc/passwd" inode=132815 dev=08
type=CWD msg=audit(1790598093.745:358): cwd="/home/kali"
type=SYSCALL msg=audit(1790598093.745:358): arch=c000003e syscall=257 success=yes exit
id=1000 sgid=1000 fsgid=1000 tty=pts0 ses=2 comm="sudo" exe="/usr/bin/sudo" subj=uncon

time→Mon Sep 28 08:21:33 2026
type=PROCTITLE msg=audit(1790598093.749:360): proctitle=2F7573722F7362696E2F756469785F
type=PATH msg=audit(1790598093.749:360): item=0 name="/etc/passwd" inode=132815 dev=08
type=CWD msg=audit(1790598093.749:360): cwd="/home/kali"
type=SYSCALL msg=audit(1790598093.749:360): arch=c000003e syscall=257 success=yes exit
=42 sgid=42 fsgid=42 tty=pts0 ses=2 comm="unix_chkpwd" exe="/usr/sbin/unix_chkpwd" sub

time→Mon Sep 28 08:21:33 2026
type=PROCTITLE msg=audit(1790598093.753:363): proctitle=7375646F00617564697463746C002D
type=PATH msg=audit(1790598093.753:363): item=0 name="/etc/passwd" inode=132815 dev=08

```

This is the one I was most interested in, since it is tied directly to the test event I just generated. The very first entries show sudo itself opening /etc/passwd (syscall 257, which is openat) - that is my nano command being picked up.

The events start at 08:21:33 on 28 September 2026. uid=1000 is my kali account, comm="sudo" and exe="/usr/bin/sudo" - so this is me, running sudo, opening /etc/passwd. A little further down, comm switches to

This proves the rule works end to end - I did something to `/etc/passwd`, and the audit system caught it, with my username, the exact time, and the exact program attached. If someone other than me had opened or edited that file, this is exactly the trail that would show it, which is the kind of evidence you'd pull during an investigation into who touched the account file and when.

Figure 9: ausearch -k auth_failures

```

kali@kali:~$ sudo asearch -k auth_failures

time--Mon Sep 28 08:21:04 2026
type=PROCTITLE msg=audit(179598064.665:177): proctitle=2F7362696E2F617564697463746C002052002F65467632F61756469742F61756469742F72756C6573
type=PATH msg=audit(179598064.665:177): item=0 name="/var/log/" inode=1966095 dev=88:01 mode=00755 ouid=0 ogid=0 rdev=000 nametype=PARENT cap_fpo=0 cap_fis=0 cap_fe=0 cap_fver=0 cap_frootid=0
type=SYSCALL msg=audit(179598064.665:177): cmd="r"
type=SOCKADDR msg=audit(179598064.665:177): saddr=1000000000000000000000000000000000
type=SYSCALL msg=audit(179598064.665:177): arch=c000003e syscall=44 success=yes exit=1088 a0=3 a1=7ffcadfe560 a2=440 a3=0 items=1 ppid=15530 pid=15550 auid=4294967295 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 s
gid=0 fsigid=0 tpe=(none) ses=4294967295 comm="auditctl" exe="/usr/sbin/auditctl" subj=unconfined key=(null)
type=CFGID_CHANGE msg=audit(179598064.665:177): auid=4294967295 ses=4294967295 subj=unconfined op=add_rule key="auth_failures" list=4 res=1

```

Timestamp 08:21:04, 28 September 2026. type=CONFIG_CHANGE, op=add_rule, comm="auditctl". The **audit** and **ses** fields both show **4294967295**, the 'unset' value, meaning no one was logged in interactively when this happened, it fired when I restarted the auditd service and it reloaded the rule, not from someone typing at a keyboard.

The rule is called `auth_failures`, but as I wrote it, it is actually just watching the `auth.log` file for writes. It is not watching for the word 'failed' or for an actual rejected login. So every record I've gotten back under this key so far is the rule being switched on, never a real authentication failure. I believe this is a real gap because if a dashboard is built off this key today, it would show zero failures, and there would be no way to tell the difference between 'nothing bad happened' and 'this has just never been tested'. So have written this up as Finding W4-F01 in Appendix A.

```
kali@kali-  
[kali@kali]~$ sudo su -  
root@kali:~# ps aux | grep program_execution  
time--Mon Sep 28 08:21:04 2026  
type=PROCTITLE msg-audit(1790598064.665:175): proctitle=F2F36269E2F617564697463746C00D252082F6574632F61756469742F61756469742F72756C6573  
type=SOCKADDR msg-audit(1790598064.665:175): saddr=1000000000000000000000000000000000  
type=SYSCALL msg-audit(1790598064.665:175): arch=c000003e syscall=44 success=yes exit=1076 a0=a3 a1=7ffcad1f660 a2=a34 a3=0 items=0 ppid=15530 pid=15550 uid=0 gid=0 euid=0 fsuid=0 egid=0  
type=EXECVE msg-audit(1790598064.665:175): argv=(null) exe="/usr/sbin/auditctl" subj=unconfined key=(null)  
type=CONFIG_CHANGE msg-audit(1790598064.665:175): add=4294967295 ses=4294967295 subj=unconfined opadd_rule key="program_execution" list=4 res=1  
  
time--Mon Sep 28 08:21:04 2026  
type=PROCTITLE msg-audit(1790598064.665:176): proctitle=F2F36269E2F617564697463746C00D252082F6574632F61756469742F61756469742F72756C6573  
type=SOCKADDR msg-audit(1790598064.665:176): saddr=1000000000000000000000000000000000  
type=SYSCALL msg-audit(1790598064.665:176): arch=c000003e syscall=44 success=yes exit=1076 a0=a3 a1=7ffcad1f660 a2=a34 a3=0 items=0 ppid=15530 pid=15550 uid=0 gid=0 euid=0 fsuid=0 egid=0  
gid=0 fsgid=0 tty=(none) ses=4294967295 comm="auditctl" exe="/usr/sbin/auditctl" subj=unconfined key=(null)  
type=CONFIG_CHANGE msg-audit(1790598064.665:176): add=4294967295 ses=4294967295 subj=unconfined opadd_rule key="program_execution" list=4 res=1  
  
time--Mon Sep 28 08:21:04 2026  
type=PROCTITLE msg-audit(1790598064.865:180): proctitle=F2F669E2F7368002F7573722F73686172652F6B616C692D74686506532F78666365342D70616E656C2D67656E6D6E2D76706E6970727368  
type=PATH msg-audit(1790598064.865:180): item=3 name="/lib64/ld-linux-x86_64.so.2" inode=1617804 dev=08:01 mode=0100755 uid=0 ogid=0 rdev=0:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_froolid=0  
type=PATH msg-audit(1790598064.865:180): item=2 name="/bin/sh" inode=1989500 dev=08:01 mode=0100755 uid=0 ogid=0 rdev=0:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_froolid=0  
type=PATH msg-audit(1790598064.865:180): item=1 name="/usr/share/kali-themes/xfce4-panel-gemmon-vpnip.sh" inode=2267297 dev=08:01 mode=0100755 uid=0 ogid=0 rdev=0:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_froolid=0  
type=PATH msg-audit(1790598064.865:180): item=0 name="/usr/share/kali-themes/xfce4-panel-gemmon-vpnip.sh" inode=2267297 dev=08:01 mode=0100755 uid=0 ogid=0 rdev=0:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_froolid=0  
type=CWD msg-audit(1790598064.865:180): cwd="/home/kali"  
type=EXECVE msg-audit(1790598064.865:180): arch=c000003e argv=(null) exe="/usr/share/kali-themes/xfce4-panel-gemmon-vpnip.sh"  
type=SYSCALL msg-audit(1790598064.865:180): argc=0 c000003e syscall=59 success=yes exit=0 a0=55b36257280 a1=55b36327d0 a2=7fff3db58168 a3=8 items=4 ppid=1354 pid=15558 uid=1000 uid=1000 gid=1000 euid=1000 fsuid=1000 egid=1000 fsgid=1000 tty=(none) ses=2 comm="xfce4-panel-gen" exe="/usr/bin/dash" subj=unconfined key="program_execution"  
  
time--Mon Sep 28 08:21:04 2026  
type=PROCTITLE msg-audit(1790598064.873:181): proctitle=6970007456E74A7E74X8007368677  
type=PATH msg-audit(1790598064.873:181): item=2 name="/lib64/ld-linux-x86_64.so.2" inode=1617804 dev=08:01 mode=0100755 uid=0 ogid=0 rdev=0:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_froolid=0  
type=PATH msg-audit(1790598064.873:181): item=1 name="/usr/bin/ip" inode=199750 dev=08:01 mode=0100777 uid=0 ogid=0 rdev=0:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_froolid=0  
type=PATH msg-audit(1790598064.873:181): item=0 name="/usr/bin/ip" inode=1999447 dev=08:01 mode=0100755 uid=0 ogid=0 rdev=0:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_froolid=0  
type=CWD msg-audit(1790598064.873:181): cwd="/home/kali"  
type=EXECVE msg-audit(1790598064.873:181): arch=c000003e argv=(null) exe="/usr/bin/tunpat" a2="show"  
type=SYSCALL msg-audit(1790598064.873:181): arch=c000003e syscall=59 success=yes exit=0 a0=55b937981fc0 a1=55b937981e50 a2=55b937981e70 a3=55b901cf6916 items=3 ppid=15559 pid=15560 uid=1000 uid=1000 gid=1000 euid=1000 fsuid=1000 egid=1000 fsgid=1000 tty=(none) ses=2 comm="ip" exe="/usr/bin/ip" subj=unconfined key="program_execution"  
  
time--Mon Sep 28 08:21:04 2026  
type=PROCTITLE msg-audit(1790598064.873:182): proctitle=637574002D6A003A00D06631  
type=PATH msg-audit(1790598064.873:182): item=2 name="/lib64/ld-linux-x86_64.so.2" inode=1617804 dev=08:01 mode=0100755 uid=0 ogid=0 rdev=0:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_froolid=0
```

Each record shows exactly which binary ran (comm/exe), the full argument list it was called with, who ran it (uid=1000, me), and whether it succeeded - genuinely useful, the kind of trail you'd need to reconstruct what happened on a machine after an incident. The one practical downside is volume: even a few minutes of normal typing

generated a lot of these records, so on a production server this rule would need a proper retention and review plan rather than someone reading it occasionally.

Figure 11: aureport

```
(kali@kali) ~$ sudo aureport
Summary Report
Range of time in logs: 09/28/2026 07:59:34.362 - 09/29/2026 16:31:40.753
Selected time for report: 09/28/2026 07:59:34 - 09/29/2026 16:31:40.753
Number of changes in configuration: 15
Number of changes to accounts, groups, or roles: 0
Number of logins: 0
Number of failed logins: 0
Number of authentications: 1
Number of failed authentications: 0
Number of users: 3
Number of terminals: 7
Number of host names: 2
Number of executables: 69
Number of commands: 85
Number of files: 118
Number of AVC's: 0
Number of MAC events: 0
Number of failed syscalls: 69
Number of anomaly events: 1
Number of responses to anomaly events: 0
Number of crypto events: 0
Number of integrity events: 0
Number of virt events: 0
Number of keys: 4
Number of process IDs: 8217
Number of events: 8676
```

The overall summary for the whole time window in the log.

Figure 12: aureport --failed

```
(kali@kali) ~$ sudo aureport --failed
Failed Summary Report
Range of time in logs: 09/28/2026 07:59:34.362 - 09/29/2026 16:32:29.475
Selected time for report: 09/28/2026 07:59:34 - 09/29/2026 16:32:29.475
Number of changes in configuration: 1
Number of changes to accounts, groups, or roles: 0
Number of logins: 0
Number of failed logins: 0
Number of authentications: 0
Number of failed authentications: 0
Number of users: 2
Number of terminals: 4
Number of host names: 1
Number of executables: 6
Number of commands: 5
Number of files: 35
Number of AVC's: 0
Number of MAC events: 0
Number of failed syscalls: 69
Number of anomaly events: 0
Number of responses to anomaly events: 0
Number of crypto events: 0
Number of integrity events: 0
Number of virt events: 0
Number of keys: 2
Number of process IDs: 17
Number of events: 70
```

Same time window, filtered to just the failure-related counts.

My Observation: Both reports agree: 0 failed logins and 0 failed authentications for the whole period, which lines up with what I found manually above. I never generated a real failed login, so there is nothing here yet. The 69 "failed syscalls" are not login failures, they are normal programs occasionally trying something that does not work, like checking for a file that isn't there, which is completely ordinary background noise. I did think it was worth a quick sanity check that none of those 69 were something like a program trying and failing to read /etc/shadow, so I have added that as a follow-up action rather than assuming it is fine.

5. Looking Through the Logs

For this part I switched from `auditd` to `journalctl` and `grep`, which look at the system from a wider angle - not just the audit rules I set up myself, but everything `systemd` and the various services on the box are logging.

5.1 Just looking around with journalctl

Figure 13: sudo journalctl (general view)

```

kali@kali:~$ sudo journalctl
[sudo] password for kali:
Feb 11 04:19:00 kali kernel: Linux version 6.16.8kali-amd64 (devel@kali.org) (x86_64-linux-gnu-gcc-14 (Debian 14.3.0-8) 14.3.0, GNU ld (GNU Binutils for Debian) 2.45) #1
Feb 11 04:19:00 kali kernel: Command line: BOOT_IMAGE=/boot/vmlinuz-6.16.8kali-amd64 root=UUID=24746b7e-1f26-43a7-8603-7350675eb65 ro quiet splash
Feb 11 04:19:00 kali kernel: BIOS-provided physical RAM map:
Feb 11 04:19:00 kali kernel: BIOS-e820: [mem 0x0000000000000000-0x000000000009f3ff] usable
Feb 11 04:19:00 kali kernel: BIOS-e820: [mem 0x000000000009f400-0x000000000009ffff] reserved
Feb 11 04:19:00 kali kernel: BIOS-e820: [mem 0x00000000000a0000-0x000000000000ffff] reserved
Feb 11 04:19:00 kali kernel: BIOS-e820: [mem 0x0000000000100000-0x0000000000fedfff] usable
Feb 11 04:19:00 kali kernel: BIOS-e820: [mem 0x0000000007fee0000-0x0000000007fefeff] ACPI data
Feb 11 04:19:00 kali kernel: BIOS-e820: [mem 0x0000000007fef0000-0x0000000007fefeff] ACPI NVS
Feb 11 04:19:00 kali kernel: BIOS-e820: [mem 0x0000000007ff00000-0x0000000007ffffff] usable
Feb 11 04:19:00 kali kernel: BIOS-e820: [mem 0x0000000008000000-0x000000000800ffff] reserved
Feb 11 04:19:00 kali kernel: BIOS-e820: [mem 0x000000000800fca0000-0x000000000800fcbfff] reserved
Feb 11 04:19:00 kali kernel: BIOS-e820: [mem 0x000000000800fcd0000-0x000000000800fdefff] reserved
Feb 11 04:19:00 kali kernel: BIOS-e820: [mem 0x000000000800ffe0000-0x000000000800ffffff] reserved
Feb 11 04:19:00 kali kernel: NX (Execute Disable) protection: active
Feb 11 04:19:00 kali kernel: APIC: Static calls initialized
Feb 11 04:19:00 kali kernel: SMBIOS 2.4 present.
Feb 11 04:19:00 kali kernel: DMI: VMware, Inc. VMware Virtual Platform/440BX Desktop Reference Platform, BIOS 6.00 03/24/2025
Feb 11 04:19:00 kali kernel: DMI: Memory slots populated: 1/128
Feb 11 04:19:00 kali kernel: vmware: hypercall mode: 0x00
Feb 11 04:19:00 kali kernel: Hypervisor detected: VMware
Feb 11 04:19:00 kali kernel: vmware: TSC freq read from hypervisor : 2099.999 Mhz
Feb 11 04:19:00 kali kernel: vmware: Host bus clock speed read from hypervisor : 660000000 Hz
Feb 11 04:19:00 kali kernel: vmware: using clock offset of 20347350402 ns
Feb 11 04:19:00 kali kernel: TSC: Detected 2099.999 Mhz processor
Feb 11 04:19:00 kali kernel: e820: update [mem 0x00000000-0x0000ffff] usable ==> reserved
Feb 11 04:19:00 kali kernel: e820: remove [mem 0x000a0000-0x0000ffff] usable
Feb 11 04:19:00 kali kernel: last_pfn = 0x80000 max_arch_pfn = 0x400000000
Feb 11 04:19:00 kali kernel: MTRR map: 7 entries (5 fixed + 2 variable; max 21), built from 8 variable MTRRs
Feb 11 04:19:00 kali kernel: x86/PAT: Configuration [0-7]: WB WC UC- WB WP UC- WT
Feb 11 04:19:00 kali kernel: Found SMP WP table at [mem 0x0006a7d0-0x0006a7f]
Feb 11 04:19:00 kali kernel: RAMDISK: [mem 0x1f5c7000-0x2badfff]
Feb 11 04:19:00 kali kernel: ACPI: Early table checksum verification disabled
Feb 11 04:19:00 kali kernel: ACPI: RSDP 0x000000000000f6A00 000024 (v02 PTLTD )
Feb 11 04:19:00 kali kernel: ACPI: XSDT 0x000000007FEEDA02 00005C (v01 INTEL 440BX 06040000 VMW 01324222)
Feb 11 04:19:00 kali kernel: ACPI: FACP 0x000000007FEFEE73 0000F4 (v04 INTEL 440BX 06040000 PTL 000F4240)
Feb 11 04:19:00 kali kernel: ACPI: DSDT 0x000000007FEFF009 010E6A (v01 PTLTD Custom 06040000 MSFT 03000001)
Feb 11 04:19:00 kali kernel: ACPI: FACS 0x000000007FEFFFC0 000040
Feb 11 04:19:00 kali kernel: ACPI: FACS 0x000000007FEFFFC0 000040
Feb 11 04:19:00 kali kernel: ACPI: BOOT 0x000000007FEEDFBC 000028 (v01 PTLTD $SBFTBL$ 06040000 LTP 00000001)
Feb 11 04:19:00 kali kernel: ACPI: APIC 0x000000007FEEDDBA 000202 (v01 PTLTD ? APIC 06040000 LTP 00000000)
Feb 11 04:19:00 kali kernel: ACPI: MCFG 0x000000007FEEDD7E 00003C (v01 PTLTD $PCITBL$ 06040000 LTP 00000001)
Feb 11 04:19:00 kali kernel: ACPI: SRAT 0x000000007FEEDAFA 000280 (v02 VMWARE MEMPLUG 06040000 VMW 00000001)

```

The very start of the journal, going right back to boot - mostly kernel messages about the hardware the VM presents (BIOS, memory map, VMware-specific detection, and so on). Nothing alarming, just background detail about the machine itself.

Figure 14: `sudo journalctl -u ssh`

```
(kali@kali)-[~]
$ sudo journalctl -u ssh
-- No entries --
```

Filtering to just the SSH service returns "-- No entries --".

That does not mean SSH is broken, rather it means the ssh unit has never logged anything on this VM, most likely because it is not running or has never been started here. On a real server that was meant to allow remote access, I would want to confirm that on purpose rather than assume, so I have added it to the event summary below as something worth a quick check.

Figure 15: `sudo journalctl --since "today"`

[illegible]

Everything logged today - a mix of scheduled daily jobs (apt-daily, logrotate, man-db), USB device detection for the virtual keyboard, and normal desktop session activity.

5.2 Checking for authentication and privilege-use activity

This VM does not keep a separate /var/log/auth.log I could read directly, so I got the same information out of journalctl instead.

Figure 16: Checking for failed passwords, then checking sudo activity

```
(kali@kali)-[~]
$ sudo journalctl --since "today" | grep -i "failed password"

(kali@kali)-[~]
$ sudo journalctl --since "today" | grep -i "sudo"
Sep 29 16:29:23 kali sudo[25583]:    kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/sbin/ausearch -k program_execution
Sep 29 16:29:23 kali sudo[25583]: pam_unix(sudo:session): session opened for user root(uid=0) by kali(uid=1000)
Sep 29 16:29:25 kali sudo[25583]: pam_unix(sudo:session): session closed for user root
Sep 29 16:30:29 kali sudo[26192]:    kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/sbin/ausearch -k auth_failures
Sep 29 16:30:29 kali sudo[26192]: pam_unix(sudo:session): session opened for user root(uid=0) by kali(uid=1000)
Sep 29 16:30:29 kali sudo[26192]: pam_unix(sudo:session): session closed for user root
Sep 29 16:30:29 kali sudo[26192]: pam_unix(sudo:session): session opened for user root
Sep 29 16:31:40 kali sudo[26821]:    kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/sbin/aureport
Sep 29 16:31:40 kali sudo[26821]: pam_unix(sudo:session): session opened for user root(uid=0) by kali(uid=1000)
Sep 29 16:31:40 kali sudo[26821]: pam_unix(sudo:session): session closed for user root
Sep 29 16:32:29 kali sudo[27232]:    kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/sbin/aureport --failed
Sep 29 16:32:29 kali sudo[27232]: pam_unix(sudo:session): session opened for user root(uid=0) by kali(uid=1000)
Sep 29 16:32:29 kali sudo[27232]: pam_unix(sudo:session): session closed for user root
Sep 29 16:33:04 kali sudo[27541]:    kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/sbin/aureport --login
Sep 29 16:33:04 kali sudo[27541]: pam_unix(sudo:session): session opened for user root(uid=0) by kali(uid=1000)
Sep 29 16:33:04 kali sudo[27541]: pam_unix(sudo:session): session closed for user root
Sep 29 16:49:47 kali sudo[35664]:    kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/bin/journalctl
Sep 29 16:49:47 kali sudo[35664]: pam_unix(sudo:session): session opened for user root(uid=0) by kali(uid=1000)
Sep 29 16:52:33 kali sudo[35664]: pam_unix(sudo:session): session closed for user root
Sep 29 16:52:42 kali sudo[37102]:    kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/bin/journalctl
Sep 29 16:52:42 kali sudo[37102]: pam_unix(sudo:session): session opened for user root(uid=0) by kali(uid=1000)
Sep 29 16:55:57 kali sudo[37102]: pam_unix(sudo:session): session closed for user root
Sep 29 16:56:13 kali sudo[38809]:    kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/bin/journalctl -u ssh
Sep 29 16:56:13 kali sudo[38809]: pam_unix(sudo:session): session opened for user root(uid=0) by kali(uid=1000)
Sep 29 16:56:13 kali sudo[38809]: pam_unix(sudo:session): session closed for user root
Sep 29 16:56:49 kali sudo[39128]:    kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/bin/journalctl --since today
Sep 29 16:56:49 kali sudo[39128]: pam_unix(sudo:session): session opened for user root(uid=0) by kali(uid=1000)
Sep 29 16:58:47 kali sudo[39128]: pam_unix(sudo:session): session closed for user root
Sep 29 16:59:09 kali sudo[40277]:    kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/bin/journalctl -p err
Sep 29 16:59:09 kali sudo[40277]: pam_unix(sudo:session): session opened for user root(uid=0) by kali(uid=1000)
Sep 29 17:02:39 kali sudo[40277]: pam_unix(sudo:session): session closed for user root
Sep 29 17:03:00 kali sudo[42141]:    kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/bin/journalctl -f
Sep 29 17:03:00 kali sudo[42141]: pam_unix(sudo:session): session opened for user root(uid=0) by kali(uid=1000)
Sep 29 17:03:33 kali sudo[42141]: pam_unix(sudo:session): session closed for user root
Sep 29 17:06:21 kali sudo[43772]:    kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/bin/journalctl
Sep 29 17:06:21 kali sudo[43772]: pam_unix(sudo:session): session opened for user root(uid=0) by kali(uid=1000)
Sep 29 17:09:35 kali sudo[43772]: pam_unix(sudo:session): session closed for user root
Sep 29 17:10:01 kali sudo[45596]:    kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/bin/journalctl --since today
Sep 29 17:10:01 kali sudo[45596]: pam_unix(sudo:session): session opened for user root
Sep 29 17:10:32 kali sudo[45855]:    kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/bin/journalctl --since today
Sep 29 17:10:32 kali sudo[45855]: pam_unix(sudo:session): session opened for user root(uid=0) by kali(uid=1000)
```

The first command (grep -i "failed password") came back with nothing at all - no failed password attempts today. The second command (grep -i "sudo") shows every single sudo command I ran, each one opening a session and closing it cleanly.

The empty result from the first command is itself useful evidence and it directly confirms I had not triggered, or had, any failed password attempt that day, which matters because it is the kind of proof I would want on file rather than just assuming it. The second command shows a completely normal admin session: me, running ausearch, aureport and journalctl, each command opening and closing a sudo session properly with no gaps or oddities. If someone else's account had shown up in that list, or a session had opened without a matching close, that's exactly the kind of thing I would chase immediately.

Figure 17: More of the sudo trail, plus a couple of other events worth a look

```

Sep 29 16:30:29 kali sudo[26193]: pam_unix(sudo:session): session opened for user root(uid=0) by kali(uid=1000)
Sep 29 16:30:29 kali sudo[26193]: pam_unix(sudo:session): session closed for user root
Sep 29 16:31:40 kali sudo[26821]: kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/sbin/aureport
Sep 29 16:31:40 kali sudo[26821]: pam_unix(sudo:session): session opened for user root(uid=0) by kali(uid=1000)
Sep 29 16:31:40 kali sudo[26821]: pam_unix(sudo:session): session closed for user root
Sep 29 16:32:29 kali sudo[27232]: kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/sbin/aureport --failed
Sep 29 16:32:29 kali sudo[27232]: pam_unix(sudo:session): session opened for user root(uid=0) by kali(uid=1000)
Sep 29 16:32:29 kali sudo[27232]: pam_unix(sudo:session): session closed for user root
Sep 29 16:33:04 kali sudo[27541]: kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/sbin/aureport --login
Sep 29 16:33:04 kali sudo[27541]: pam_unix(sudo:session): session opened for user root(uid=0) by kali(uid=1000)
Sep 29 16:33:04 kali sudo[27541]: pam_unix(sudo:session): session closed for user root
Sep 29 16:35:01 kali CRON[28479]: pam_unix(cron:session): session opened for user root(uid=0) by root(uid=0)
Sep 29 16:35:01 kali CRON[28481]: (root) CMD (command -v debian-sa1 > /dev/null && debian-sa1 1 1)
Sep 29 16:35:01 kali CRON[28479]: pam_unix(cron:session): session closed for user root
Sep 29 16:39:01 kali CRON[30405]: pam_unix(cron:session): session opened for user root(uid=0) by root(uid=0)
Sep 29 16:39:01 kali CRON[30407]: (root) CMD ( [ -x /usr/lib/php/sessionclean ] && if [ ! -d /run/systemd/system ]; then /usr/lib/php/sessionclean; fi)
Sep 29 16:39:01 kali CRON[30405]: pam_unix(cron:session): session closed for user root
Sep 29 16:39:12 kali systemd[1]: Starting phpsessionclean.service - Clean php session files...
Sep 29 16:39:13 kali systemd[1]: phpsessionclean.service: Deactivated successfully.
Sep 29 16:39:13 kali systemd[1]: Finished phpsessionclean.service - Clean php session files.
Sep 29 16:39:28 kali auditd[15526]: Audit daemon rotating log files
Sep 29 16:44:47 kali NetworkManager[664]: <info> [1790714687.1652] dhcp4 (eth0): state changed new lease, address=192.168.233.128
Sep 29 16:44:47 kali dbus-daemon[585]: [system] Activating via systemd: service name='org.freedesktop.nm_dispatcher' unit='dbus-org.freedesktop.nm-dispatcher.service' requested by: root
Sep 29 16:44:47 kali systemd[1]: Starting NetworkManager-dispatcher.service - Network Manager Script Dispatcher Service...
Sep 29 16:44:47 kali dbus-daemon[585]: [system] Successfully activated service 'org.freedesktop.nm_dispatcher'
Sep 29 16:44:47 kali systemd[1]: Started NetworkManager-dispatcher.service - Network Manager Script Dispatcher Service.
Sep 29 16:44:57 kali systemd[1]: NetworkManager-dispatcher.service: Deactivated successfully.
Sep 29 16:45:01 kali CRON[33331]: pam_unix(cron:session): session opened for user root(uid=0) by root(uid=0)
Sep 29 16:45:01 kali CRON[33333]: (root) CMD (command -v debian-sa1 > /dev/null && debian-sa1 1 1)
Sep 29 16:45:01 kali CRON[33331]: pam_unix(cron:session): session closed for user root
Sep 29 16:49:20 kali dbus-daemon[1156]: [session uid=1000 pid=1156 pidfd=5] Activating via systemd: service name='org.xfce.Xfconf' unit='xfconfd.service' requested by: root
Sep 29 16:49:20 kali systemd[1133]: Starting xfconfd.service - Xfce configuration service...
Sep 29 16:49:20 kali dbus-daemon[1156]: [session uid=1000 pid=1156 pidfd=5] Successfully activated service 'org.xfce.Xfconf'
Sep 29 16:49:20 kali systemd[1133]: Started xfconfd.service - Xfce configuration service.
Sep 29 16:49:26 kali xfce4-screensaver-dialog[35407]: gkr-pam: unlocked login keyring
Sep 29 16:49:26 kali xfce4-screensaver-dialog[35404]: pam_unix(xfce-screensaver:account): setuid failed: Operation not permitted
Sep 29 16:49:47 kali sudo[35664]: kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/bin/journalctl
Sep 29 16:49:47 kali sudo[35664]: pam_unix(sudo:session): session opened for user root(uid=0) by kali(uid=1000)
Sep 29 16:52:33 kali sudo[35664]: pam_unix(sudo:session): session closed for user root
Sep 29 16:52:42 kali sudo[37102]: kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/bin/journalctl
Sep 29 16:52:42 kali sudo[37102]: pam_unix(sudo:session): session opened for user root(uid=0) by kali(uid=1000)
Sep 29 16:54:46 kali auditd[15526]: Audit daemon rotating log files
Sep 29 16:55:01 kali CRON[38218]: pam_unix(cron:session): session opened for user root(uid=0) by root(uid=0)
Sep 29 16:55:01 kali CRON[38220]: (root) CMD (command -v debian-sa1 > /dev/null && debian-sa1 1 1)
Sep 29 16:55:01 kali CRON[38218]: pam_unix(cron:session): session closed for user root
lines 15323-15368/15368 (END)

```

Further down the same grep, I can see scheduled cron jobs running as root, a DHCP lease renewal, and one line that caught my eye: `xfce4-screensaver-dialog` reporting "setuid failed: Operation not permitted".

My honest read on the screensaver line is that it is most likely a desktop-permissions quirk specific to this VM rather than a real security issue, since the dialog otherwise seems to work fine (the line just above it shows the login keyring unlocking normally). But because it touches a security-relevant piece of software, the screen lock, I think it deserves a one-line check rather than being ignored, so I've added it to the event summary below as a low-priority item to confirm.

5.3 Checking errors and warnings

Figure 18: `sudo journalctl -p err` (errors only, part 1)

```

(kali@kali): ~
$ sudo journalctl -p err
Feb 11 04:19:00 kali kernel: p1lx4_smbus 0000:00:07.3: SMBus Host Controller not enabled!
Feb 11 04:21:35 kali lightdm[1302]: gkr-pam: unable to locate daemon control file
Feb 11 04:21:46 kali oboxd[1812]: Unable to acquire registry: Error calling StartServiceByName for org.gnome.evolution.dataserver.Sources5: Unit evolution-source-registry not found.
Feb 11 04:21:46 kali oboxd[1812]: Unable to acquire registry: Error calling StartServiceByName for org.gnome.evolution.dataserver.Sources5: Unit evolution-source-registry not found.
-- Boot 44a57f5f4d4e4a9e356263752c8a0 --
Feb 11 04:57:49 kali kernel: p1lx4_smbus 0000:00:07.3: SMBus Host Controller not enabled!
Feb 11 04:58:18 kali lightdm[1160]: gkr-pam: unable to locate daemon control file
Feb 11 04:58:26 kali oboxd[1733]: Unable to acquire registry: Error calling StartServiceByName for org.gnome.evolution.dataserver.Sources5: Unit evolution-source-registry not found.
Feb 11 04:58:26 kali oboxd[1733]: Unable to acquire registry: Error calling StartServiceByName for org.gnome.evolution.dataserver.Sources5: Unit evolution-source-registry not found.
Feb 11 04:58:55 kali systemd[1]: Failed to start NetworkManager-wait-online.service - Network Manager Wait Online.
-- Boot f17e88a2f1140afae6eb72fa76828f4c --
Feb 11 05:26:38 kali kernel: p1lx4_smbus 0000:00:07.3: SMBus Host Controller not enabled!
Feb 11 05:26:38 kali (generato[378]: /usr/lib/systemd/system-generators/systemd-sshd-generator terminated by signal ABRT.
Feb 11 05:27:16 kali lightdm[1114]: gkr-pam: unable to locate daemon control file
Feb 11 05:27:27 kali oboxd[1710]: Unable to acquire registry: Error calling StartServiceByName for org.gnome.evolution.dataserver.Sources5: Unit evolution-source-registry not found.
Feb 11 05:27:27 kali oboxd[1710]: Unable to acquire registry: Error calling StartServiceByName for org.gnome.evolution.dataserver.Sources5: Unit evolution-source-registry not found.
Feb 11 05:27:41 kali systemd[1]: Failed to start NetworkManager-wait-online.service - Network Manager Wait Online.
-- Boot 50e9ab5dfabf48949d4b34a5cc5f5e21 --
Jul 14 04:56:05 kali kernel: p1lx4_smbus 0000:00:07.3: SMBus Host Controller not enabled!
Jul 14 04:56:05 kali (generato[378]: /usr/lib/systemd/system-generators/systemd-sshd-generator terminated by signal ABRT.
Jul 14 04:56:52 kali lightdm[1342]: gkr-pam: unable to locate daemon control file
Jul 14 04:56:58 kali oboxd[1902]: Unable to acquire registry: Error calling StartServiceByName for org.gnome.evolution.dataserver.Sources5: Unit evolution-source-registry not found.
Jul 14 04:56:58 kali oboxd[1902]: Unable to acquire registry: Error calling StartServiceByName for org.gnome.evolution.dataserver.Sources5: Unit evolution-source-registry not found.
-- Boot 44a8f93ad6584bcbad67b55ab89f6ae --
Jul 15 04:42:42 kali kernel: p1lx4_smbus 0000:00:07.3: SMBus Host Controller not enabled!
Jul 15 04:43:21 kali lightdm[1209]: gkr-pam: unable to locate daemon control file
Jul 15 04:43:32 kali oboxd[1798]: Unable to acquire registry: Error calling StartServiceByName for org.gnome.evolution.dataserver.Sources5: Unit evolution-source-registry not found.
Jul 15 04:43:32 kali oboxd[1798]: Unable to acquire registry: Error calling StartServiceByName for org.gnome.evolution.dataserver.Sources5: Unit evolution-source-registry not found.
Jul 15 07:22:57 kali systemd-networkd-wait-online[48300]: Timeout occurred while waiting for network connectivity.
Jul 16 06:11:07 kali systemd-networkd-wait-online[83831]: Timeout occurred while waiting for network connectivity.
Jul 16 06:36:23 kali systemd-networkd-wait-online[96925]: Timeout occurred while waiting for network connectivity.
-- Boot eb6c414258574743a8ddddd28155a71 --
Jul 22 06:44:56 kali kernel: p1lx4_smbus 0000:00:07.3: SMBus Host Controller not enabled!
Jul 22 06:45:17 kali lightdm[1173]: gkr-pam: unable to locate daemon control file
Jul 22 06:45:27 kali oboxd[1754]: Unable to acquire registry: Error calling StartServiceByName for org.gnome.evolution.dataserver.Sources5: Unit evolution-source-registry not found.
Jul 22 06:45:27 kali oboxd[1754]: Unable to acquire registry: Error calling StartServiceByName for org.gnome.evolution.dataserver.Sources5: Unit evolution-source-registry not found.
-- Boot 656779616d774f42ba20fd6d583c260d7 --
Sep 28 07:55:05 kali kernel: p1lx4_smbus 0000:00:07.3: SMBus Host Controller not enabled!
Sep 28 07:55:33 kali lightdm[1109]: gkr-pam: unable to locate daemon control file
Sep 28 07:55:18 kali oboxd[1736]: Unable to acquire registry: Error calling StartServiceByName for org.gnome.evolution.dataserver.Sources5: Unit evolution-source-registry not found.
Sep 28 07:55:18 kali oboxd[1736]: Unable to acquire registry: Error calling StartServiceByName for org.gnome.evolution.dataserver.Sources5: Unit evolution-source-registry not found.
Sep 28 08:00:05 kali (generato[4750]: /usr/lib/systemd/system-generators/systemd-sshd-generator terminated by signal SEGV.
Sep 29 16:28:13 kali systemd-networkd-wait-online[24719]: Timeout occurred while waiting for network connectivity.
lines 1-43/43 (END)

```

This one surprised me - it shows the same handful of errors repeating across several different boots going back to July: a virtual hardware controller that is never enabled, a couple of desktop services that can not find files they are looking for, and the one that stood out - a component called `systemd-sshd-generator` that was "terminated by signal ABRT" on one boot and, as the next screenshot shows, actually crashed with a segfault on another.

Figure 19: `sudo journalctl -p err` (errors only, part 2)

```

kali@kali: ~
Jul 14 04:56:10 kali polkitd[640]: Error opening rules directory: Error opening directory "/usr/local/share/polkit-1/rules.d": No such file or directory (g-file-error-quark, 4)
Jul 14 04:56:15 kali wireplumber[1255]: default: Failed to get percentage from UPower: org.freedesktop.DBus.Error.NameHasNoOwner
Jul 14 04:56:53 kali wireplumber[1394]: default: Failed to get percentage from UPower: org.freedesktop.DBus.Error.NameHasNoOwner
Jul 14 04:56:58 kali obexd[1902]: Unable to acquire registry: Error calling StartServiceByName for org.gnome.evolution.dataserver.Sources5: Unit evolution-source-registry.service not found.
Jul 14 04:56:58 kali obexd[1902]: Unable to acquire registry: Error calling StartServiceByName for org.gnome.evolution.dataserver.Sources5: Unit evolution-source-registry.service not found.
Jul 15 04:42:42 kali kernel: RAS: Correctable Errors collector initialized.
Jul 15 04:42:52 kali polkitd[565]: Error opening rules directory: Error opening directory "/run/polkit-1/rules.d": No such file or directory (g-file-error-quark, 4)
Jul 15 04:42:52 kali polkitd[565]: Error opening rules directory: Error opening directory "/usr/local/share/polkit-1/rules.d": No such file or directory (g-file-error-quark, 4)
Jul 15 04:42:58 kali wireplumber[1130]: default: Failed to get percentage from UPower: org.freedesktop.DBus.Error.NameHasNoOwner
Jul 15 04:43:23 kali wireplumber[1260]: default: Failed to get percentage from UPower: org.freedesktop.DBus.Error.NameHasNoOwner
Jul 15 04:43:32 kali obexd[1798]: Unable to acquire registry: Error calling StartServiceByName for org.gnome.evolution.dataserver.Sources5: Unit evolution-source-registry.service not found.
Jul 15 04:43:32 kali obexd[1798]: Unable to acquire registry: Error calling StartServiceByName for org.gnome.evolution.dataserver.Sources5: Unit evolution-source-registry.service not found.
Jul 15 07:22:57 kali apt-helper[48298]: E: Sub-process /lib/systemd/systemd-networkd-wait-online returned an error code (1)
Jul 16 06:11:07 kali apt-helper[83762]: E: Sub-process /lib/systemd/systemd-networkd-wait-online returned an error code (1)
Jul 16 06:36:23 kali apt-helper[96923]: E: Sub-process /lib/systemd/systemd-networkd-wait-online returned an error code (1)
Jul 22 06:44:56 kali kernel: RAS: Correctable Errors collector initialized.
Jul 22 06:44:59 kali polkitd[631]: Error opening rules directory: Error opening directory "/run/polkit-1/rules.d": No such file or directory (g-file-error-quark, 4)
Jul 22 06:44:59 kali polkitd[631]: Error opening rules directory: Error opening directory "/usr/local/share/polkit-1/rules.d": No such file or directory (g-file-error-quark, 4)
Jul 22 06:45:06 kali wireplumber[1100]: default: Failed to get percentage from UPower: org.freedesktop.DBus.Error.NameHasNoOwner
Jul 22 06:45:18 kali wireplumber[1224]: default: Failed to get percentage from UPower: org.freedesktop.DBus.Error.NameHasNoOwner
Jul 22 06:45:27 kali obexd[1754]: Unable to acquire registry: Error calling StartServiceByName for org.gnome.evolution.dataserver.Sources5: Unit evolution-source-registry.service not found.
Jul 22 06:45:27 kali obexd[1754]: Unable to acquire registry: Error calling StartServiceByName for org.gnome.evolution.dataserver.Sources5: Unit evolution-source-registry.service not found.
Sep 28 07:55:05 kali kernel: RAS: Correctable Errors collector initialized.
Sep 28 07:55:07 kali polkitd[591]: Error opening rules directory: Error opening directory "/run/polkit-1/rules.d": No such file or directory (g-file-error-quark, 4)
Sep 28 07:55:07 kali polkitd[591]: Error opening rules directory: Error opening directory "/usr/local/share/polkit-1/rules.d": No such file or directory (g-file-error-quark, 4)
Sep 28 07:55:18 kali wireplumber[1035]: default: Failed to get percentage from UPower: org.freedesktop.DBus.Error.NameHasNoOwner
Sep 28 07:55:34 kali wireplumber[1161]: default: Failed to get percentage from UPower: org.freedesktop.DBus.Error.NameHasNoOwner
Sep 28 07:55:18 kali obexd[1736]: Unable to acquire registry: Error calling StartServiceByName for org.gnome.evolution.dataserver.Sources5: Unit evolution-source-registry.service not found.
Sep 28 07:55:18 kali obexd[1736]: Unable to acquire registry: Error calling StartServiceByName for org.gnome.evolution.dataserver.Sources5: Unit evolution-source-registry.service not found.
Sep 28 08:00:05 kali kernel: systemd-sshd-ge[4768]: segfault at 0 ip 00007f412f429f29 sp 00007fcd18c9598 error 6 in libc.so.6[184f29,7f412f2cd000+16a000] likely on CPU 0 (core 0, socket 0)
Sep 29 16:28:13 kali apt-helper[24714]: E: Sub-process /lib/systemd/systemd-networkd-wait-online returned an error code (1)

```

Scrolling further, I found the actual crash: "systemd-sshd-ge[4768]: segfault at 0 ... in libc.so.6 ... likely on CPU 0", dated 28 September.

Most of what is on this list is background noise VMs generate all the time - missing optional hardware, desktop components that can not find an icon cache, that sort of thing. The sshd-generator crash is different: it is an actual segfault, it has happened more than once across separate boots, and nothing on this system had caught or alerted on it before I went looking. sshd is normally used to let one port serve more than one protocol, for example sharing port 443 between HTTPS and SSH, so if it's supposed to be doing that job here and its generator keeps crashing, that's a genuine reliability and potentially security-relevant gap, not just cosmetic noise. I have written this up as Finding W4-F04.

Figure 20: `sudo journalctl --since "today" | grep -i "error" / "warning"`

```

kali@kali: ~
$ sudo journalctl --since "today" | grep -i "error"
Sep 29 16:28:13 kali apt-helper[24714]: E: Sub-process /lib/systemd/systemd-networkd-wait-online returned an error code (1)

kali@kali: ~
$ sudo journalctl --since "today" | grep -i "warning"

kali@kali: ~
$

```

One error today (an unrelated apt-helper/ networked timing issue), and zero warnings.

5.4 Event summary

Event / Condition	Evidence Source	Time	User / Service	Why it matters	What I'd do about it
No failed password attempts found	journalctl, grep "failed password"	29 Sep 2026 (today's log)	N/A - no event	Confirms nothing suspicious happened with logins today, which is good, but it is also the only day I checked	Keep checking this daily rather than assuming it stays clean
Full sudo trail captured cleanly	journalctl, grep "sudo"	29 Sep, 16:29-17:10	kali (elevating to root)	Shows every privileged command is logged and matches what I actually did - no gaps, no unexpected user	None - this is working as it should
systemd-sshd-generator crashed (segfault)	journalctl -p err	28 Sep, 08:00:05 (also seen on an earlier boot)	systemd-sshd-generator service	A service generator crashing could mean sshd is not providing whatever protocol-sharing it's meant to, and no one had noticed	Look into why it is crashing; check the sshd package/config; watch for it happening again
xfce4-screensaver-dialog: setuid failed	journalctl (sudo/pam trail)	29 Sep, 16:49:26	kali, xfce4-screensaver-dialog	Low risk, but it touches the screen-lock, so worth a one-off check	Confirm the screen actually still locks properly; if yes, no further action

Event / Condition	Evidence Source	Time	User / Service	Why it matters	What I'd do about it
journalctl -u ssh returned no entries	journalctl -u ssh	29 Sep (time of check)	sshd (unit)	Means SSH has never logged anything here - could just mean it is not used on this VM, but worth confirming on purpose	Check whether SSH is meant to be running on this host
NetworkManager-wait-online timing out repeatedly	journalctl -p err	Recurring, Jul-Sep 2026	systemd-networkd-wait-online	Not a security issue by itself, but the same error has gone unaddressed for months, which says something about how closely errors get reviewed	Low priority - could disable the wait-online unit if it is not actually needed

6. Running a Hardening Scan with Lynis

Lynis was already available on this VM (version 3.1.6), so I confirmed that and went straight into running a full scan with the `sudo lynis audit system`.

Figure 21: Checking the Lynis version and starting the scan

```
(kali@kali)~$ lynis --version
3.1.6

(kali@kali)~$ which lynis
/usr/sbin/lynis

(kali@kali)~$ sudo lynis audit system

[ Lynis 3.1.6 ]

=====
Lynis comes with ABSOLUTELY NO WARRANTY. This is free software, and you are
welcome to redistribute it under the terms of the GNU General Public license.
See the LICENSE file for details about using this software.
=====

2007-2025, CISofy - https://cisofy.com/lynis/
Enterprise support available (compliance, plugins, interface and tools)
=====

[+] Initializing program
- Detecting OS ... [ DONE ]
- Checking profiles ... [ DONE ]

Program version: 3.1.6
Operating system: Linux
Operating system name: Kali Linux
Operating system version: Rolling release
End-of-life:
Kernel version: 6.10.8-kali
Hardware platform: x86_64
Hostname: kali

Profiles: /etc/lynis/default.prfl
Log file: /var/log/lynis.log
Report file: /var/log/lynis-report.dat
Report version: 1.0
Plugin directory: /etc/lynis/plugins

Auditor: [Not Specified]
Language: en
```

`lynis --version` confirms 3.1.6, which lines up with where the binary lives (`/usr/sbin/lynis`). Then the `sudo lynis audit system` kicks off the scan.

Figure 22: Warnings and the start of the suggestions list

```
Warnings (1):
- Couldn't find 2 responsive nameservers [NETW-2705]
  https://cisofy.com/lynis/controls/NETW-2705/

Suggestions (48):
* Install libpam-tmpdir to set $TMP and $TMPDIR for PAM sessions [DEB-0280]
- Related resources
* Website: https://cisofy.com/lynis/controls/DEB-0280/

* Install apt-listbugs to display a list of critical bugs prior to each APT installation. [DEB-0810]
- Related resources
* Website: https://cisofy.com/lynis/controls/DEB-0810/

* Install apt-listchanges to display any significant changes prior to any upgrade via APT. [DEB-0811]
- Related resources
* Website: https://cisofy.com/lynis/controls/DEB-0811/

* Install needrestart, alternatively to debian-goodies, so that you can run needrestart after upgrades to determine which daemons are using old versions of libraries and
need restarting. [DEB-0831]
- Related resources
* Website: https://cisofy.com/lynis/controls/DEB-0831/

* Install fail2ban to automatically ban hosts that commit multiple authentication errors. [DEB-0880]
- Related resources
* Website: https://cisofy.com/lynis/controls/DEB-0880/
```

One warning: Lynis could not find two responsive nameservers. Then 48 suggestions in total - I have picked out the ones I think matter most below.

Figure 23: Kernel hardening comparison

```
[+] Kernel Hardening

- Comparing sysctl key pairs with scan profile
- dev.tty.ldisc_autoload (exp: 0) [ DIFFERENT ]
- fs.protected_fifos (exp: 2) [ DIFFERENT ]
- fs.protected_hardlinks (exp: 1) [ OK ]
- fs.protected_regular (exp: 2) [ OK ]
- fs.protected_symlinks (exp: 1) [ OK ]
- fs.suid_dumpable (exp: 0) [ OK ]
- kernel.core_uses_pid (exp: 1) [ OK ]
- kernel.ctrl-alt-del (exp: 0) [ OK ]
- kernel.dmesg_restrict (exp: 1) [ DIFFERENT ]
- kernel.kptr_restrict (exp: 2) [ DIFFERENT ]
- kernel.modules_disabled (exp: 1) [ DIFFERENT ]
- kernel.perf_event_paranoid (exp: 2 3 4) [ OK ]
- kernel.randomize_va_space (exp: 2) [ OK ]
- kernel.sysrq (exp: 0) [ DIFFERENT ]
- kernel.unprivileged_bpf_disabled (exp: 1) [ DIFFERENT ]
- kernel.yama.ptrace_scope (exp: 1 2 3) [ DIFFERENT ]
- net.core.bpf_jit_harden (exp: 2) [ DIFFERENT ]
- net.ipv4.conf.all.accept_redirects (exp: 0) [ DIFFERENT ]
- net.ipv4.conf.all.send_redirects (exp: 0) [ OK ]
- net.ipv4.conf.all.bootp_relay (exp: 0) [ OK ]
- net.ipv4.conf.all.forwarding (exp: 0) [ OK ]
- net.ipv4.conf.all.log_martians (exp: 1) [ DIFFERENT ]
- net.ipv4.conf.all.mc_forwarding (exp: 0) [ OK ]
- net.ipv4.conf.all.proxy_arp (exp: 0) [ OK ]
- net.ipv4.conf.all.rp_filter (exp: 1) [ DIFFERENT ]
- net.ipv4.conf.all.send_redirects (exp: 0) [ DIFFERENT ]
- net.ipv4.conf.default.accept_redirects (exp: 0) [ DIFFERENT ]
- net.ipv4.conf.default.accept_source_route (exp: 0) [ OK ]
- net.ipv4.conf.default.log_martians (exp: 1) [ DIFFERENT ]
- net.ipv4.icmp_echo_ignore_broadcasts (exp: 1) [ OK ]
- net.ipv4.icmp_ignore_bogus_error_responses (exp: 1) [ OK ]
- net.ipv4.tcp_syncookies (exp: 1) [ OK ]
- net.ipv4.tcp_timestamps (exp: 0 1) [ OK ]
- net.ipv6.conf.all.accept_redirects (exp: 0) [ DIFFERENT ]
- net.ipv6.conf.all.accept_source_route (exp: 0) [ OK ]
- net.ipv6.conf.default.accept_redirects (exp: 0) [ DIFFERENT ]
- net.ipv6.conf.default.accept_source_route (exp: 0) [ OK ]

[+] Hardening
- Installed compiler(s) [ FOUND ]
- Installed malware scanner [ NOT FOUND ]
```

Lynis compares a long list of kernel and network settings against what it considers a hardened baseline. A lot of them came back "DIFFERENT" rather than "OK" - things like `kernel.dmesg_restrict`, `kernel.kptr_restrict`, `kernel.yama.ptrace_scope`, and several `net.ipv4/net.ipv6` redirect and logging settings.

Figure 24: The final summary

```

Details:
Hardening index : 61 [##### ]
Tests performed : 272
Plugins enabled : 1

Software components:
- Firewall           [ ]
- Intrusion software [ ]
- Malware scanner    [ ]

Files:
- Test and debug information : /var/log/lynis.log
- Report data                : /var/log/lynis-report.dat

Notice: This version of Lynis is older than 6 months and might be outdated. Check the project page if a newer version is available.

Notice: No OS entry was found in the end-of-life database

What to do:
Please submit a pull request on GitHub to include your OS version and the end date of this OS version is being supported
URL: https://github.com/CISOfy/lynis

Lynis 3.1.6
Auditing, system hardening, and compliance for UNIX-based systems
(Linux, macOS, BSD, and others)
2007-2025, CISOfy - https://cisofy.com/lynis/
Enterprise support available (compliance, plugins, interface and tools)

[TIP]: Enhance Lynis audits by adding your settings to custom.prfl (see /etc/lynis/default.prfl for all settings)

```

Hardening index: 61 out of 100, from 272 tests. Under Software components, Firewall, Intrusion software, and Malware scanner are all marked with a red X - none of the three are installed.

6.1 The top findings

Suggestion / Finding	What Lynis said	Risk area	Likely owner	What I'd recommend	Priority
No firewall installed	Software components: Firewall - not found	Nothing is filtering inbound/outbound traffic at all	System/Linux Administrator	Install and enable a firewall (ufw or nftables would both be reasonable choices) with a sensible default-deny inbound policy	High
No malware scanner installed	Software components: Malware scanner - not found	Nothing on the host would catch a dropped malicious file	System/Linux Administrator	Install a scanner appropriate to the host's purpose (for example ClamAV for a general-purpose box)	High
fail2ban not installed [DEB-0880]	Suggestion: install fail2ban to automatically ban hosts that commit multiple authentication errors	No automatic brute-force protection on authentication	System/Linux Administrator	Install fail2ban and point it at whichever authentication log this VM actually uses	High - connects directly to the auth_failures rule that has never been tested
GRUB has no boot-loader password [BOOT-5122]	Suggestion: set a password on GRUB to prevent altering boot configuration	Anyone with physical or console access could change how the machine boots	System/Linux Administrator	Set a GRUB password, store it somewhere safe	Moderate (matters more on a physical or shared machine, less in an isolated lab VM)
Many kernel/network hardening settings differ from the hardened baseline	Kernel Hardening section - multiple "DIFFERENT" results (e.g. <code>kernel.dmesg_restrict</code> , <code>kernel.kptr_restrict</code> , <code>net.ipv4</code> redirect and logging settings)	Weaker resistance to kernel information leaks and certain network-level attacks than a hardened baseline would give	System/Linux Administrator, with sign-off from whoever owns the security baseline	Go through the differences one at a time, apply the ones that make sense for this host, and document any the organisation deliberately doesn't want	Moderate
No responsive nameservers found [NETW-2705]	Warning (not a suggestion) - couldn't find 2 responsive nameservers	Could affect reliability of name resolution, which some security tools depend on	System/Linux Administrator	Check the VM's DNS configuration	Low

6.2 Fixing one of them myself and proving it worked

I wanted to actually fix one low-risk suggestion myself and show a before-and-after, so I picked apt-listchanges (suggestion DEB-0811) - it just shows a summary of what's changing before an upgrade goes ahead, which felt like a genuinely low-risk, sensible thing to turn on.

Figure 25: Before: apt-listchanges wasn't installed

```
(kali@kali)-[~]
$ dpkg -l apt-listchanges
dpkg-query: no packages found matching apt-listchanges
```

dpkg -l apt-listchanges confirms: "no packages found matching apt-listchanges".

Figure 26: Installing it

```
(kali@kali)-[~]
$ sudo apt update
Hit:1 http://http.kali.org/kali kali-rolling InRelease
2240 packages can be upgraded. Run 'apt list --upgradable' to see them.

(kali@kali)-[~]
$ sudo apt install apt-listchanges
The following package was automatically installed and is no longer required:
  libcrypt-dev
Use 'sudo apt autoremove' to remove it.

Upgrading:
  debconf debconf-i18n

Installing:
  apt-listchanges

Installing dependencies:
  python3-debconf

Suggested packages:
  default-mta | mail-transport-agent

Summary:
  Upgrading: 2, Installing: 2, Removing: 0, Not Upgrading: 2238
  Download size: 495 kB
  Space needed: 606 kB / 61.3 GB available

Continue? [Y/n] Y
Get:1 http://kali.download/kali kali-rolling/main amd64 debconf-i18n all 1.5.92 [217 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 debconf all 1.5.92 [123 kB]
Get:3 http://kali.download/kali kali-rolling/main amd64 python3-debconf all 1.5.92 [4,252 B]
Get:4 http://http.kali.org/kali kali-rolling/main amd64 apt-listchanges all 4.8+nmu3 [151 kB]
Fetched 495 kB in 15s (33.0 kB/s)
```

sudo apt install apt-listchanges pulls in the package and a couple of small dependencies.

Figure 27: After: confirming it's actually installed

```
Get:1 http://kali.download/kali kali-rolling/main amd64 debconf-i18n all 1.5.92 [217 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 debconf all 1.5.92 [123 kB]
Get:3 http://kali.download/kali kali-rolling/main amd64 python3-debconf all 1.5.92 [4,252 B]
Get:4 http://http.kali.org/kali kali-rolling/main amd64 apt-listchanges all 4.8+nmu3 [151 kB]
Fetched 495 kB in 15s (33.0 kB/s)
Preconfiguring packages ...
Deferring configuration of apt-listchanges until /usr/bin/python3
and python's debconf module are available
(Reading database ... 423680 files and directories currently installed.)
Preparing to unpack .../debconf-i18n_1.5.92_all.deb ...
Unpacking debconf-i18n (1.5.92) over (1.5.91) ...
Preparing to unpack .../debconf_1.5.92_all.deb ...
Unpacking debconf (1.5.92) over (1.5.91) ...
Setting up debconf (1.5.92) ...
Selecting previously unselected package python3-debconf.
(Reading database ... 423682 files and directories currently installed.)
Preparing to unpack .../python3-debconf_1.5.92_all.deb ...
Unpacking python3-debconf (1.5.92) ...
Selecting previously unselected package apt-listchanges.
Preparing to unpack .../apt-listchanges_4.8+nmu3_all.deb ...
Unpacking apt-listchanges (4.8+nmu3) ...
Setting up python3-debconf (1.5.92) ...
Setting up debconf-i18n (1.5.92) ...
Setting up apt-listchanges (4.8+nmu3) ...
Creating config file /etc/apt/listchanges.conf with new version
Created symlink '/etc/systemd/system/timers.target.wants/apt-listchanges.timer' → '/usr/lib/systemd/system/apt-listchanges.timer'.
apt-listchanges.service is a disabled or a static unit, not starting it.
Processing triggers for man-db (2.13.1-1) ...
Processing triggers for kali-menu (2025.4.3) ...

(kali@kali)-[~]
$ dpkg -l apt-listchanges
Desired=Unknown/Install/Remove/Purge/Hold
| Status=Not/Inst/Conf-files/Unpacked/halF-conf/Half-inst/trig-aWait/Trig-pend
|/ Err?=(none)/Reinst-required (Status,Err: uppercase=bad)
++-- Name -- Version -- Architecture -- Description
++--
ii apt-listchanges 4.8+nmu3 all package change history notification tool

(kali@kali)-[~]
$
```

dpkg -l apt-listchanges now shows "ii" next to it, which means it's properly installed and configured.

Figure 28: Re-running Lynis to prove the fix worked

```

Suggestions (46):
* Install libpam-tmpdir to set $TMP and $TMPDIR for PAM sessions [DEB-0280]
  - Related resources
    * Website: https://cisofy.com/lynis/controls/DEB-0280/
* Install apt-listbugs to display a list of critical bugs prior to each APT installation. [DEB-0810]
  - Related resources
    * Website: https://cisofy.com/lynis/controls/DEB-0810/
* Install needrestart, alternatively to debian-goodies, so that you can run needrestart after upgrades to determine which daemons are using old versions of libraries and need restarting. [DEB-0831]
  - Related resources
    * Website: https://cisofy.com/lynis/controls/DEB-0831/
* Install fail2ban to automatically ban hosts that commit multiple authentication errors. [DEB-0880]
  - Related resources
    * Website: https://cisofy.com/lynis/controls/DEB-0880/
* Set a password on GRUB boot loader to prevent altering boot configuration (e.g. boot in single user mode without password) [BOOT-5122]
  - Related resources
    * Website: https://cisofy.com/lynis/controls/BOOT-5122/
* Determine runlevel and services at startup [BOOT-5180]
  - Related resources
    * Website: https://cisofy.com/lynis/controls/BOOT-5180/

```

I ran the scan again and looked at the suggestions list - DEB-0811 (the apt-listchanges suggestion) is gone, and the total suggestion count dropped from 48 to 46.

The point of doing this was not the fix itself, it is proving I can close the loop: find a real issue, fix it properly, and then use the same tool to independently confirm the fix actually worked, rather than just assuming it did.

7. Control-Monitoring Table

Pulling everything together, here are the five controls I'd actually track going forward, all built from evidence I collected in this lab rather than anything hypothetical.

Control / Objective	Evidence	Owner	Observed Status	KPI/KRI or Threshold	Risk / Significance	Remediation	Retest / Follow-up
auditd is installed, running, and its rules are loaded	Figures 1-6	System/ Linux Administrator	Green - confirmed active (running) and all five rules loaded	KPI: auditd active = yes, checked at every review	Low now that it's confirmed, but worth a quick check at every review since a silent stop would otherwise go unnoticed	None needed right now	Re-check systemctl status auditd at each review to confirm it's stayed active
Authentication failures are actually detected	Figure 9 (ausearch -k auth_failures)	System Administrator to fix; Security / Assurance to verify	Not proven - only config-change events so far, no real failure captured	KRI: a real test failure is captured within [x] minutes	High - I can't currently trust this control to catch a brute-force attempt	Run one instructor-approved test failed login and confirm it's captured; longer term, watch the actual authentication events, not just the log file	Re-run ausearch -k auth_failures after a controlled test and confirm a genuine failure record appears
Basic host protection software is present (firewall, IDS, malware scanner)	Figure 24 (software components)	System/ Linux Administrator	Red - all three missing	KPI: all three installed and enabled = yes	High - nothing is actively watching or blocking anything on this host right now	Install a firewall with default-deny inbound, an IDS or file-integrity tool, and a malware scanner	Re-run the scan; confirm all three show as found/enabled
Kernel and network settings match the hardened baseline	Figure 23 (Kernel Hardening comparison)	System/ Linux Administrator	Amber - many settings flagged DIFFERENT	KPI: % of checked settings matching baseline	Moderate - weaker resistance to some kernel and network-level attacks than intended	Work through the differences, apply what's appropriate, document any deliberate exceptions	Re-run the scan; check the DIFFERENT count has gone down
Critical services stay up without crashing	Figures 18-19 (sslh-generator segfault)	System/ Linux Administrator	Amber - repeated crash across at least two boots, unnoticed until this review	KRI: 0 unexplained service crashes per week	Moderate - a crashing security-relevant service could quietly stop doing its job	Investigate and fix the sslh-generator crash; check package/config integrity	Check journalctl -p err after the fix; confirm no further segfault over the following week
Change hygiene is improving (apt-listchanges)	Figures 25-28	System/ Linux Administrator	Green - fixed and independently confirmed	N/A - already closed	Low - this one's done	None needed	Already retested; suggestion confirmed gone from the output

8. How This Would Feed a SIEM

I did all of this by hand on one VM, but in a real environment it would need to feed a SIEM watching lots of machines at once. Here is how I would map each piece across, and where I think the line sits between "just log it" and "actually wake someone up".

What I collected here	How it would reach a SIEM	Stays a routine technical alert when...	Becomes a governance/escalation issue when...
auditd events (rule changes, file watches, program execution)	A forwarding plugin (like audisp-syslog) ships these to syslog, which an agent forwards on to the SIEM	A normal, expected rule change made through proper change control - like the ones I made in this lab	The audit service itself gets disabled, or a rule gets removed, outside of any approved change - since auditd is itself one of the things I'm supposed to be monitoring
The auth_failures gap I found	Once actually fixed to watch real authentication events (not just the log file), forwarded the same way	A one-off, explainable failed login (someone mistyped a password)	Repeated failures on one account, failures spread across many accounts, or any failure on a privileged account - classic brute-force pattern, should go straight to the SOC
The missing firewall/IDS/malware scanner	Not something you "forward" - it's more that a SIEM or asset-management tool should flag hosts missing required protections as a compliance gap	N/A - a missing control isn't really an alert, it's a standing gap	This should already be a governance issue the moment it's found, which is exactly why it's my top finding above
The sslh-generator crash	Forwarded as a standard system-error log entry	A single, one-off crash with an obvious cause	The same service keeps crashing repeatedly with no fix - which is exactly the pattern I found, so I'd treat this as ready to escalate, not just watch
Lynis hardening index and suggestions	Not real-time - more something a scheduled job re-runs periodically and a dashboard tracks the trend of, the same way I'd track any KPI	The index holds steady or improves over time	The index drops, or the same high-priority suggestion (like fail2ban or the firewall) stays open for months without action

Almost everything I found that actually mattered was not a single dramatic event - it was something that had been quietly sitting there unnoticed (the sslh crash, the untested auth rule, the missing firewall) until I specifically went looking. That is really the whole point of continuous monitoring: a SIEM's job is not just to catch the big obvious attack, it is to make sure the quiet stuff like this does not sit unnoticed for months the way it clearly had here.

9. Applied Governance Task From Linux Evidence to Control Assurance

Question	My answer
1. Which finding is the highest priority, and why?	The missing firewall/IDS/malware scanner (Finding W4-F02). It's the most fundamental gap - there's currently nothing actively watching or blocking anything on this host. It's made worse by the fact that my one authentication-detection rule has also never been proven to work (Finding W4-F01), so right now a real attack could get through and go undetected at the same time.
2. Who should own remediation for each major finding?	For all four of my findings, the actual fix (installing software, patching config, investigating the crash) sits with whoever administers this Linux host day to day - the System/Linux Administrator. But proving the fix actually works - re-running the checks, confirming the evidence - is the Security Governance and Assurance function's job, not the administrator's own word for it. I've kept that split in the control table above.
3. What should trigger escalation to the CISO or a management committee?	Based on what I found, I'd escalate when: any finding I've rated High stays open past its target date, a security-relevant service (like ssh here) crashes more than once without a fix, or the Lynis hardening index drops or stays flat for two scans in a row. Basically: anything that shows a control isn't just temporarily behind, but isn't being followed up on at all.
4. What evidence would show a finding is actually fixed?	The same kind of evidence I used for my own apt-listchanges fix: a before state, the actual change, and then an independent re-check using the same tool that found the problem in the first place - not just someone saying it's done. For the auth_failures gap specifically, that means a real ausearch record showing an actual failure, not another config-change entry.
5. When and how should each control be retested?	Straight away after the fix, the same way I retested apt-listchanges. After that, on a regular cycle - I'd suggest auditd's status and rules get checked at every review, Lynis gets re-run monthly, and anything that was rated High gets specifically re-checked at the very next review rather than waiting for the next full cycle.

10. Remediation and Retest Plan

#	Finding	Priority	Action	Owner	Target	How I'd know it's fixed
1	No firewall, IDS, or malware scanner (W4-F02)	High	Install and enable a firewall with default-deny inbound; install a file-integrity/IDS tool and a malware scanner	System/Linux Administrator	Before next review	Re-run Lynis; all three show as found/enabled in Software components
2	auth_failures rule never proven (W4-F01)	High	Run one instructor-approved test failed login; confirm it's captured; longer term, watch real auth events, not just the log file	System Administrator (fix); Security/Assurance (verify)	Before next review	ausearch -k auth_failures shows a genuine failure record
3	systemd-ssh-generator crashing (W4-F04)	Moderate	Investigate the crash (check package/config); apply a fix; watch for recurrence	System/Linux Administrator	Within 2 weeks	journalctl -p err shows no further segfault from this service over the following week
4	Kernel/network hardening gaps and no GRUB password (W4-F03)	Moderate	Review each DIFFERENT setting and apply what's appropriate; set a GRUB password	System/Linux Administrator	Within 30 days	Re-run Lynis; fewer DIFFERENT results, hardening index has gone up
5	apt-listchanges suggestion (DEB-0811)	Done	Installed and independently re-verified	Me	Complete	Already retested - confirmed gone from the scan output

11. Conclusion

Going into this, I expected the technical part (installing things, running commands) to be the hard part, and the governance write-up to just be paperwork on top. It turned out the other way round - actually running the commands was straightforward, but working out which results genuinely mattered took a lot more thought than I expected.

The main thing I'll take away is that a control existing and a control actually working are two different things, and you can only tell them apart by testing. My `auth_failures` rule looked exactly right sitting in the rules file - it only became obvious it wasn't doing its job when I actually went looking at what it had captured. The same goes for things like the `ssllh` crash: it was sitting in the logs the whole time, it just hadn't been looked at.

Overall, I'd describe this VM's security posture as **partially set up but not yet proven**. The pieces are mostly there - `auditd` works, `Lynis` runs cleanly, I can see exactly what's happening on the box - but a few of the things that are supposed to actually protect or detect (the firewall, the malware scanner, the `auth_failures` rule) aren't actually doing their job yet. The good news is that where I did fix something myself, I could prove it worked, and I have tried to set the rest of this report up so the same close-the-loop approach can be applied to everything I've flagged.

Appendix A: Finding Records

W4-F01

Date / Time Observed	28 September 2026, 08:21:04 (rule created); still unproven as of 29 September 2026, 16:32 (aureport still shows 0 failed logins/authentications)
Evidence Source	auditd (ausearch -k auth_failures, Figure 9; aureport / aureport --failed, Figures 11-12)
Evidence Summary	I set up a rule watching /var/log/auth.log under the key auth_failures. Every record I've gotten back under that key is a CONFIG_CHANGE event (the rule being created or the daemon restarting) - never a real authentication failure. aureport confirms 0 failed logins and 0 failed authentications across the whole period I checked.
Security Significance	A rule named for catching authentication failures that has never actually caught one gives false confidence - a dashboard built on this key would read "zero failures" whether that means the system is secure or just that the detection has never been tested.
Control Objective	Detect and log failed authentication attempts so brute-force or credential-stuffing activity can be spotted and investigated.
Control Owner	System/Linux Administrator (to fix); Security Governance and Assurance (to independently verify)
Risk / Priority	High
Recommended Action	Run one instructor-approved test failed login on this VM and confirm ausearch captures it. Longer term, watch the actual PAM/authentication events rather than only the log file's attribute changes.
Retest Evidence	ausearch -k auth_failures showing a genuine failure record, not just a CONFIG_CHANGE entry

W4-F02

Date / Time Observed	29 September 2026, during the Lynis scan
Evidence Source	Lynis (Figure 24 - Software components)
Evidence Summary	The Lynis scan marked Firewall, Intrusion software, and Malware scanner as all missing. Lynis also separately suggested installing fail2ban [DEB-0880], which is directly related.
Security Significance	This is the biggest gap I found. There is currently nothing actively filtering traffic, watching for intrusions, or scanning for malware on this host - and it directly compounds W4-F01, since there's also no automatic response to repeated authentication failures.
Control Objective	Provide a baseline layer of active protection (blocking, detection, scanning) on the host, not just logging after the fact.
Control Owner	System/Linux Administrator
Risk / Priority	High
Recommended Action	Install and enable a firewall with a default-deny inbound policy, an intrusion-detection or file-integrity tool, a malware scanner, and fail2ban.
Retest Evidence	Re-run Lynis and confirm all software components show as found/enabled

W4-F03

Date / Time Observed	29 September 2026, during the Lynis scan
Evidence Source	Lynis (Figure 23 - Kernel Hardening comparison; also BOOT-5122 GRUB suggestion)
Evidence Summary	A number of kernel and network sysctl settings (for example kernel.dmesg_restrict, kernel.kptr_restrict, kernel.yama.ptrace_scope, and several net.ipv4/net.ipv6 redirect and logging settings) came back "DIFFERENT" rather than "OK" against Lynis's hardened baseline. Separately, Lynis noted there's no password set on GRUB.
Security Significance	These settings individually aren't dramatic, but together they mean the system gives up some resistance to kernel information leaks, certain network redirect attacks, and (for the GRUB gap) unauthorised changes to how the machine boots if someone has physical or console access.
Control Objective	Bring the host's kernel and network configuration in line with an agreed hardened baseline, and prevent unauthorised changes to boot configuration.
Control Owner	System/Linux Administrator, with sign-off from whoever owns the security baseline for this environment
Risk / Priority	Moderate
Recommended Action	Go through the DIFFERENT settings one at a time, apply the ones appropriate to this host, and document any the organisation deliberately keeps different. Set a GRUB password.
Retest Evidence	Re-run Lynis; fewer DIFFERENT results and the GRUB suggestion no longer appears

W4-F04

Date / Time Observed	28 September 2026, 08:00:05 (also a similar failure on an earlier boot, 22 July 2026)
Evidence Source	journalctl -p err (Figures 18-19)
Evidence Summary	systemd-ssh-generator crashed with a segfault ("segfault at 0 ... in libc.so.6 ... likely on CPU 0") on 28 September, and was separately reported as "terminated by signal ABRT" on an earlier boot on 22 July. This had not been picked up or investigated before I found it while reviewing error-priority logs.
Security Significance	ssh is normally used to share one network port between more than one protocol. If its generator keeps crashing, whatever it's meant to be doing may not actually be happening, and the fact this went unnoticed across two separate boots shows error logs aren't being reviewed regularly.
Control Objective	Security-relevant services should run reliably, and crashes should be noticed and investigated close to when they happen, not months later.
Control Owner	System/Linux Administrator
Risk / Priority	Moderate
Recommended Action	Investigate the cause of the crash (check the ssh package version and configuration), apply a fix, and keep an eye on the logs afterwards to confirm it doesn't happen again.
Retest Evidence	journalctl -p err shows no further segfault or abort from this service over the following week

Appendix B: Screenshot Index

Figure	What it shows	Section
1	systemctl status auditd; apt update; apt install auditd	4.1
2	apt install (continued); systemctl start/enable auditd	4.1
3	systemctl status auditd, confirming active (running)	4.1
4	nano custom.rules (typing the rules)	4.2
5	cat custom.rules (confirming save)	4.2
6	systemctl restart auditd; auditctl -l	4.2
7	nano /etc/passwd (no save); ls /tmp	4.3
8	ausearch -k passwd_changes	4.3
9	ausearch -k auth_failures	4.3
10	ausearch -k program_execution	4.3
11	aureport	4.3
12	aureport --failed	4.3
13	journalctl (general)	5.1
14	journalctl -u ssh	5.1
15	journalctl --since today	5.1
16	grep "failed password"; grep "sudo"	5.2
17	grep "sudo" (continued)	5.2
18	journalctl -p err (part 1)	5.3
19	journalctl -p err (part 2, sslh segfault)	5.3
20	grep "error"; grep "warning"	5.3
21	lynis --version; lynis audit system (start)	6
22	Warnings + Suggestions list	6
23	Kernel Hardening comparison	6
24	Final summary (hardening index 61)	6
25	dpkg -l apt-listchanges (before)	6.2
26	apt install apt-listchanges	6.2
27	dpkg -l apt-listchanges (after)	6.2
28	Lynis re-run (DEB-0811 gone)	6.2

All screenshots came from my own assigned Kali VM. I have cropped a few to cut blank space, and blacked out the interactive sudo password prompt line in three places (Figures 1, 10 and 26) as a precaution, even though no password was ever actually visible in the originals.